

1. 가상화 기술과 컨테이너 가상화

도커(Docker)는 컨테이너를 생성·실행·관리하는 소프트웨어다.
컨테이너(Container) : 하나의 서버를 효율적으로 나눠 쓰기 위한
가상화 기술로, 애플리케이션을 격리된 환경에서 실행하는 단위다.

도커를 제대로 이해하려면 **서버와 가상화 기술**에 대한 배경지식이 필요하다. 도커를 본격적으로 공부하기 전에 두 개념을 간략하게 살펴본다.

1.1 서버

서버(Server)는 **서비스를 제공하는 컴퓨터**를 의미하며, 하드웨어와 소프트웨어를 모두 포함하는 개념이다. 즉, 무언가를 **제공하는 주체**이다.

1.1.1 서버와 클라이언트

서버로부터 무언가를 받으려면 원하는 것을 서버에 **요청(Request)** 해야 한다. 그리고 서버는 이 요청에 대한 **응답(Response)** 을 제공한다. 여기서 서버에게 원하는 것을 요청하는 주체를 **클라이언트(Client)** 라고 한다.

Client —(Request)—▶ Server —(Response)—▶
Client

1.1.2 서버의 종류

대표적으로 파일 서버, DB 서버, 웹 서버, 웹 애플리케이션 서버 등이 있다.

종류	역할	대표 소프트웨어
파일 서버	파일 공유 소프트웨어가 실행되어 파일을 저장·공유하는 서버	Samba, NFS, FTP 서버
DB 서버	DBMS가 설치되어 데이터를 저장·조회하는 서버	MySQL, PostgreSQL, Oracle
웹 서버 (Web Server)	정적 콘텐츠(HTML, CSS, JS, 이미지) 제공에 특화된 서버	Apache HTTP Server, Nginx
웹 애플리케이션 서버 (WAS)	비즈니스 로직을 실행해 동적 콘텐츠를 생성하는 서버	Tomcat, Jetty, Undertow

Web Server

- 정적 콘텐츠에 특화되어 있다.
- 처리 흐름 : 요청 분석 → 해당 정적 파일 탐색 → 파일 읽기 → 클라이언트에 전송
- 별다른 계산 로직 없이 네트워크 요청·응답 처리에 집중한다.
- 가볍고 빠르며, 대표적으로 **Apache HTTP Server, Nginx** 등이 있다.

Web Application Server (WAS)

- 로그인, DB 연동, 세션 처리 등 **애플리케이션 로직**을 수행할 수 있다.
- 서블릿 컨테이너(Servlet Container) 역할을 하며, 대표적으로 **Tomcat, Jetty, Undertow** 등이 있다. Spring Boot는 내장 Tomcat 위에서 애플리케이션을 실행한다.
- 처리 흐름 : 요청 수신 → DB와 통신 → 결과 처리 → 동적으로 HTML(또는 JSON) 생성 후 응답

왜 웹 서버와 WAS를 분리할까?

정적 리소스는 웹 서버가 직접 빠르게 처리해 **WAS의 부하를 줄이고**, 동적 요청은 웹 서버가 **리버스 프록시(Reverse Proxy)**로서

WAS에게 전달해 WAS가 애플리케이션 로직 처리를 담당하도록 한다.

이 밖에도 로드 밸런싱, SSL 처리, 보안(WAS를 외부에 직접 노출하지 않음) 측면의 이점이 있다.

1.1.3 엔터프라이즈 환경의 서버 운영

기업에서 여러 서버를 운영하는 방법은 크게 베어메탈, 하이퍼바이저, 컨테이너 세 가지가 있다.

1.1.3.1 베어메탈

베어메탈(Bare Metal)은 가상화 계층 없이 물리 서버에 OS를 직접 설치해 사용하는 방식이다. 우리가 일반적으로 PC에 OS를 설치하고 소프트웨어를 실행하는 것과 유사하다. 즉, 물리 서버 1대에 OS를 설치하고 그 OS에서 여러 소프트웨어를 함께 운영한다.

1.1.3.2 하이퍼바이저와 컨테이너 방식

두 방식 모두 서버 운영에 가상화 기술을 활용한다. 물리 서버의 수나 실행되는 소프트웨어는 베어메탈과 동일하지만, 중간에 VM(Virtual Machine)이나 컨테이너라는 계층이 추가된 형태이다.

1.2 가상화 기술

먼저 가상이란 무엇인지 생각해보자. 가상은 "실제로 존재하지 않지만 마치 존재하는 것처럼 느껴지는 것"이라고 볼 수 있다. 이 개념을 서버에 적용한 것이 가상화 기술이다. 즉, 실제 물리 서버가 따로 존재하지 않더라도 마치 서버가 있는 것처럼 만드는 기술이다.

즉, 1대의 물리 서버에서 여러 대의 (논리적인) 서버를 동시에 실행하는 것이다.

1.2.1 가상화 기술과 소프트웨어

베어메탈 방식은 하나의 운영체제에서 모든 프로그램을 실행하므로, 리소스만 충분하다면 관리하기 쉽다. 하지만 다음과 같은 문제가 있다.

- **보안 문제** : 프로그램 하나가 해킹되면 같은 OS에서 실행 중인 다른 프로그램의 정보까지 탈취될 위험이 있다.
- **리소스 독점 문제** : 한 프로그램의 리소스 사용량이 급증해 모든 리소스를 소모하면, 나머지 프로그램이 정상적으로 실행되지 않을 수 있다.
- **의존성 충돌 문제** : 프로그램마다 요구하는 라이브러리·런타임 버전이 다르면 한 OS 안에서 충돌이 발생할 수 있다.

가상화 기술을 사용하면 한 대의 물리 컴퓨터에 **여러 개의 논리적인 실행 환경**을 구성할 수 있다. 그리고 각 가상 환경마다 **사용 가능한 최대 리소스를 할당**할 수 있다.



가상 환경은 논리적으로 격리되어 있어 한 프로그램에 오류가 생겨도 다른 프로그램에 영향을 주지 않는다. 또한 특정 프로그램의 리소스 사용량이 급증하더라도 **할당된 범위 내로 리소스 사용량이 제한**되므로 다른 환경을 침범하지 않는다.

1.2.2 가상화 기술의 경제성

"독립된 OS 환경에서 소프트웨어를 운영하는 것이 안전하다면, 차라리 하드웨어를 여러 대 사용하는 것이 더 간단하지 않을까? 왜 굳이 가상화 기술을 사용할까?" 라는 질문을 할 수 있다.

기술이 발전할수록 하드웨어 성능은 향상되고, 소프트웨어 하나가 요구하는 리소스는 상대적으로 작아진다. 저사양 서버 여러 대보다 **고사양 서버 한 대를 나눠 쓰는 편이 비용, 설치 공간, 인력, 운영 측면에서 더 경제**적이다. 서버를 여러 대 운영하면 전력 소비와 관리 비용도 함께 증가한다.

💡 서버 한 대에 서비스 하나만 올리면 대부분의 시간 동안 CPU·메모리가 놀게 된다. 가상화는 이렇게 남는 자원을 여러 환경에 나눠 활용률을 높이는 기술이기도 하다.

1.3 하이퍼바이저 가상화

하이퍼바이저 가상화는 초기부터 사용된 전통적인 가상화 방식이다. **하이퍼바이저(Hypervisor)**는 가상 머신(VM)을 생성·실행하고 관리하는 소프트웨어다.

서버에 하이퍼바이저를 설치하면 **가상 환경을 생성**하고, CPU나 메모리 같은 **리소스를 할당**할 수 있다. 하이퍼바이저로 생성한 가상 환경은 디스크 공간을 차지하며, 실행 중인 가상 환경은 할당받은 만큼의 리소스를 사용할 수 있다.

하이퍼바이저는 설치 위치에 따라 두 가지 유형으로 나뉜다.

구분	Type 1 (네이티브 / 베어메탈형)	Type 2 (호스트형)
설치 위치	물리 하드웨어 위에 직접 설치	기존 OS(Host OS) 위에 애플리케이션처럼 설치
특징	중간 OS가 없어 오버헤드가 적고 성능이 좋음	설치·사용이 간편하지만 Host OS를 거쳐 상대적으로 느림
주 용도	데이터센터, 클라우드 서버	개인 PC, 개발·테스트 환경
예시	VMware ESXi, Xen, Microsoft Hyper-V, KVM	VirtualBox, VMware Workstation, Parallels

이 글에서 설명하는 **Host OS** 위에서 하이퍼바이저를 실행하는 구조는 **Type 2**에 해당한다.

하이퍼바이저를 실행하는 OS를 **Host OS**라고 부른다. Host OS는 물리적인 하드웨어 서버를 관리한다. 하이퍼바이저는 **Host OS가 관리하는**

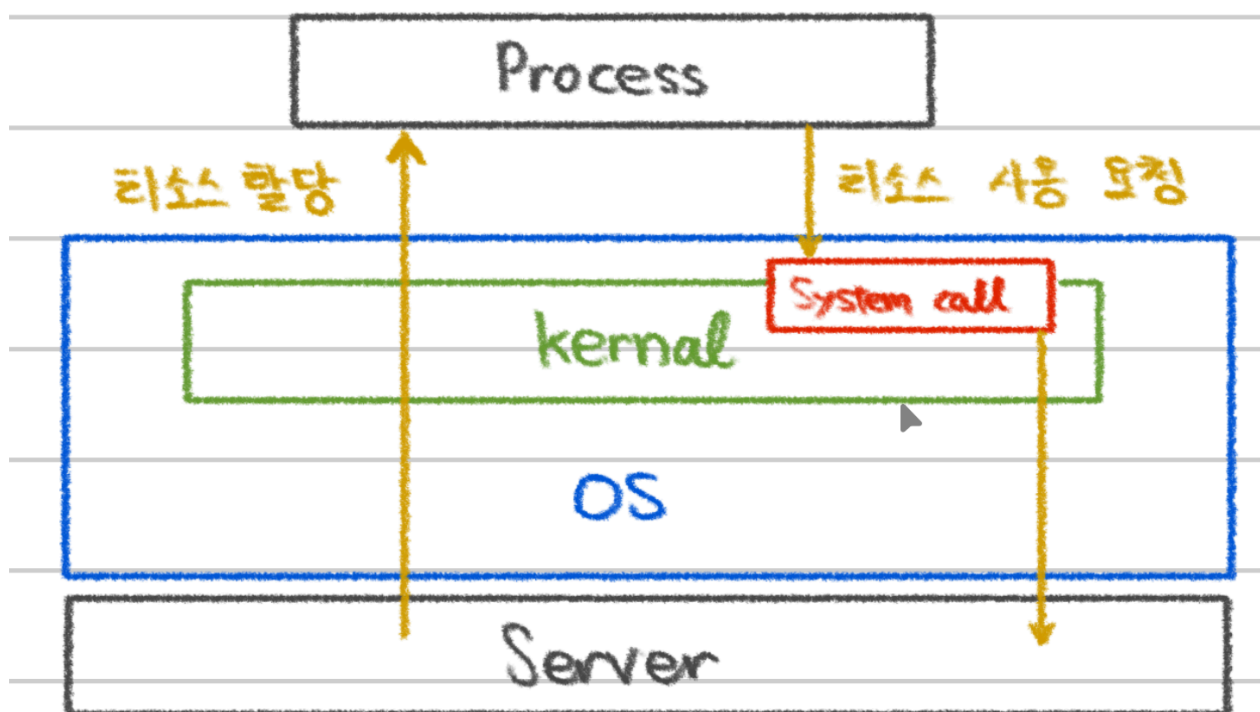
자원을 나눠 새로운 가상 OS인 Guest OS를 실행한다.

- Host OS : 물리 서버를 직접 관리하는 OS
- Guest OS : Host OS(하이퍼바이저)가 나눠준 리소스를 사용하는 논리적인 공간에서 실행되는 OS

논리적으로 분리되어 Guest OS가 실행되는 가상의 컴퓨터를 **Virtual Machine(VM)** 이라고 부른다.

1.3.1 프로세스와 OS

하이퍼바이저의 작동 원리를 이해하려면, 이미 알고 있을 수도 있지만 **프로세스와 운영체제의 관계를 먼저 짚고 넘어가야 한다.**



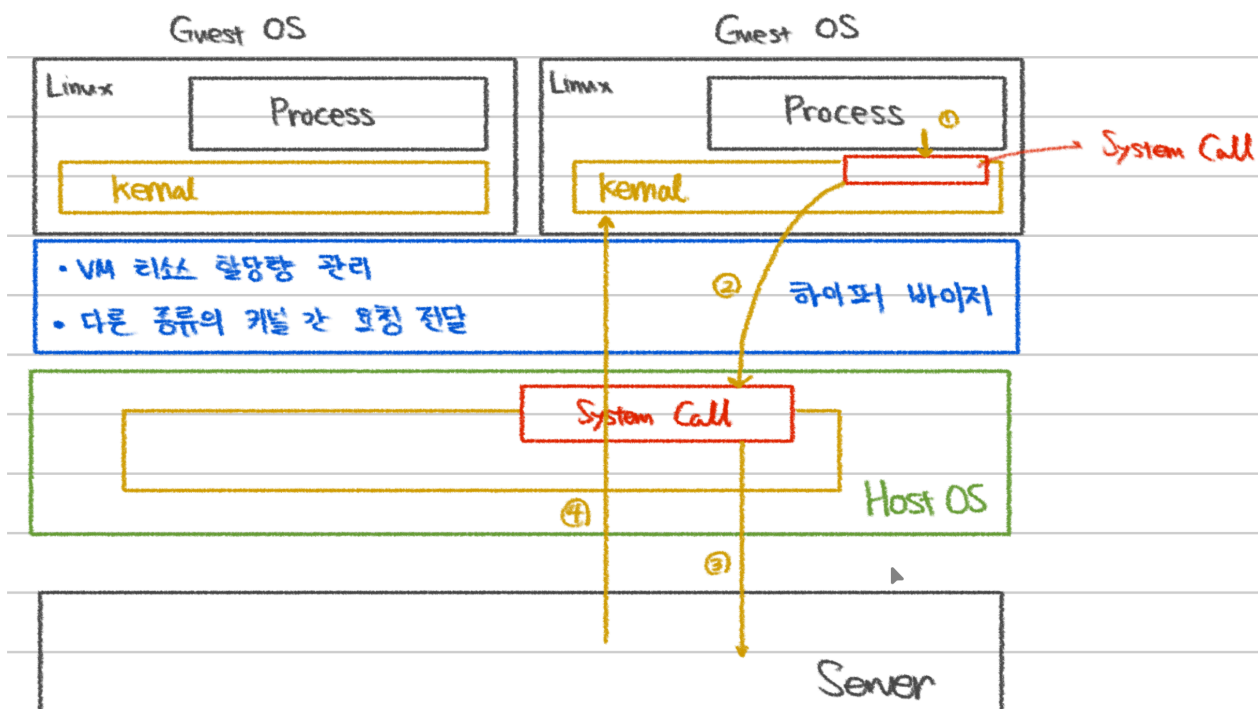
프로세스를 실행하려면 CPU나 메모리 같은 하드웨어 리소스가 필요하다. 하지만 프로세스는 하드웨어에 직접 접근할 수 없고, **운영체제에 리소스 사용을 요청**해야 한다. OS에는 이러한 요청을 처리하는 **커널 (Kernel)** 이 존재하며, 커널은 프로세스의 요청을 받기 위한 인터페이스로 **시스템 콜(System Call)** 을 제공한다.

프로세스 —(System Call)—▶ 커널 —▶ 하드웨어(CPU, 메모리, 디스크 등)

1.3.2 하이퍼바이저의 역할

각각의 OS는 고유한 커널과 시스템 콜을 사용한다. 예를 들어 Linux용 프로그램은 Linux 커널의 시스템 콜을, Windows용 프로그램은 Windows 커널의 시스템 콜을 사용한다.

아래 그림은 Windows Host OS에서 Linux를 Guest OS로 실행하는 구조이다.



Guest OS 내부의 프로세스가 보낸 시스템 콜은 **Guest OS의 커널**이 처리한다. 문제는 Guest OS의 커널이 실제로 다룰 물리 하드웨어를 갖고 있지 않다는 점이다. Guest 커널은 자신이 진짜 하드웨어 위에서 동작한다고 생각하고 하드웨어에 접근하려 한다.

이때 하이퍼바이저가 중간에서 이 하드웨어 접근을 가로채 실제 물리 자원으로 연결해준다. 하이퍼바이저는 각 VM에게 가상의 **CPU**, 메모리, 디스크, 네트워크 카드를 제공하고, Guest 커널의 하드웨어 요청을 실제 하드

웨어(Type 2라면 Host OS를 거쳐)에 맞게 변환·중재한다. 덕분에 Host OS와 종류가 다른 OS(예: Windows 위의 Linux)도 Guest OS로 실행할 수 있다.

즉, 하이퍼바이저는 Guest OS와 실제 하드웨어 사이에서 **통역사이자 중재자** 역할을 하며 둘 사이의 상호작용을 가능하게 한다.

하이퍼바이저는 여러 VM(각자 다른 커널)의 하드웨어 요청을 관리하고, 각 VM에 필요한 리소스를 할당한다. 참고로 최근 CPU는 **Intel VT-x, AMD-V** 같은 하드웨어 가상화 지원 기능을 제공해, 이 중재 과정의 성능 저하를 크게 줄여준다.

하이퍼바이저 방식의 단점 VM마다 **OS 전체(커널 포함)**를 따로 설치해야 하므로 이미지 용량이 크고(수 GB), 부팅에 시간이 걸리며, OS 자체가 사용하는 메모리·CPU 오버헤드가 발생한다.

1.4 컨테이너 가상화

컨테이너 가상화는 현대 애플리케이션 운영 환경에서 선호되는 기술이다. Guest OS를 통째로 띄우지 않기 때문에 하이퍼바이저 가상화보다 **가볍고 빠르다**.

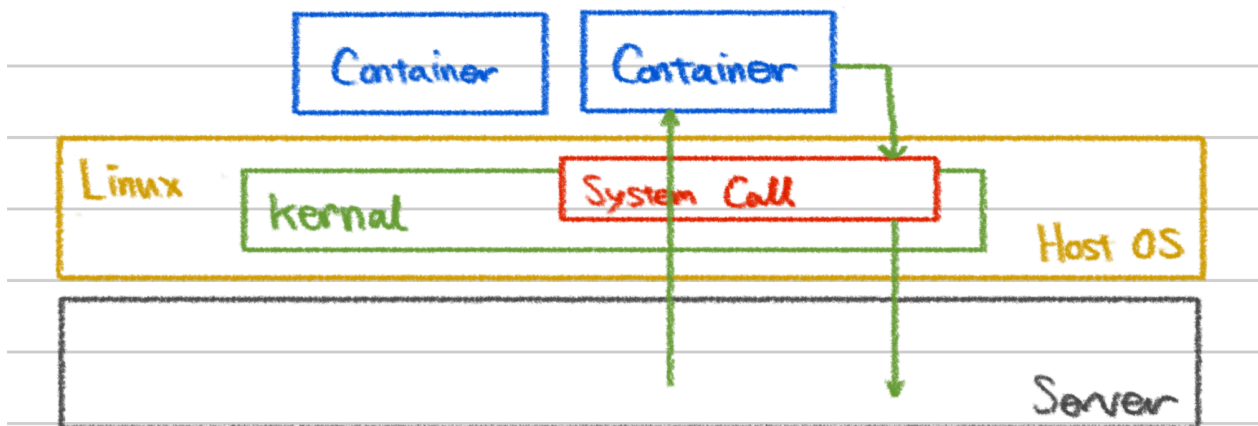
컨테이너 가상화는 **LXC(Linux Containers)**를 통해 본격적으로 대중화되었다. LXC는 하이퍼바이저 같은 별도의 가상화 계층 없이, **리눅스 커널이 제공하는 기능만으로 격리된 공간을 만드는 도구**이다. 여기서 사용되는 핵심 커널 기능은 다음 두 가지다.

- **Namespace** : 프로세스(PID), 파일시스템(Mount), 네트워크(NET), 호스트명(UTS), IPC, 사용자(USER) 등을 분리해 "**무엇을 볼 수 있는가**"를 격리한다.

- **cgroup (Control Groups)** : CPU, 메모리, 디스크 I/O 등의 리소스 사용량을 제한하고 관리한다. 즉, "얼마나 쓸 수 있는가"를 제한한다.

그리고 이렇게 격리된 공간을 컨테이너라고 한다.

📝 도커도 초기(2013년)에는 LXC를 기반으로 컨테이너를 실행했다. 이후 자체 라이브러리인 `libcontainer` (현재의 `runc`)로 전환했지만, **Namespace**와 **cgroup**을 사용한다는 핵심 원리는 동일하다.



- 모든 컨테이너가 **Host OS**의 커널 하나를 공유한다.
- 컨테이너 가상화 : 하이퍼바이저 없이 **커널의 기능을 사용하는** 가상화 기술 (OS 수준 가상화라고도 한다)

컨테이너는 커널의 격리 기능을 활용하며, **모든 컨테이너가 Host OS의 커널을 공유**한다. 이는 컨테이너 가상화의 가장 중요한 특징이다. 컨테이너 입장에서는 독립된 OS처럼 보이지만, Host OS 입장에서는 **격리된 프로세스**일 뿐이다.

커널을 공유한다는 특징 때문에 다음과 같은 장단점이 생긴다.

- Guest OS 부팅이 필요 없어 수 초 이내로 실행되고, 이미지 용량도 수십~수백 MB 수준으로 작다.
- OS 오버헤드가 적어 같은 서버에서 **VM보다 훨씬 많은 컨테이너**를 실행할 수 있다.
- 커널을 공유하므로 **Host OS와 다른 커널을 요구하는 OS는 실행할 수 없다.** (예: Linux 호스트에서 Windows 컨테이너 실행 불가)
- 커널이 뚫리면 모든 컨테이너가 영향을 받을 수 있어, **격리 수준은 VM보다 약하다.**

****Windows·macOS에서는 어떻게 리눅스 컨테이너를 실행할까?**

Docker Desktop은 내부적으로 경량 Linux VM(Windows는 WSL 2, macOS는 가상화 프레임워크)을 하나 띄우고, 그 Linux 커널 위에서 컨테이너를 실행한다. 결국 리눅스 컨테이너는 리눅스 커널이 있어야 동작한다.

하이퍼바이저 vs 컨테이너 한눈에 보기

구분	하이퍼바이저 가상화 (VM)	컨테이너 가상화
가상화 대상	하드웨어	OS(커널) 수준
커널	VM마다 독립된 커널 보유	Host OS 커널 공유
실행 속도	느림 (OS 부팅 필요, 수십 초)	빠름 (프로세스 실행, 수 초 이내)
용량	큼 (수 GB)	작음 (수십)
격리 수준	강함	상대적으로 약함
다른 OS 실행	가능 (Windows 위 Linux 등)	Host 커널과 호환되는 환경만 가능
대표 기술	VMware, VirtualBox, KVM, Hyper-V	LXC, Docker, containerd, Podman